

EC 504
Fall, 2025
Exam 1

Thursday, Oct. 23, 2025

Last Name	First Name	Student ID #
-----------	------------	--------------

Honor Code: This exam represents only my own work. I did not give or receive help.

Signature: _____

Instructions

- Print your name and BU ID on the first page carefully so automatic text recognition will identify you.
- This exam allows you to use 3 double sided pages of notes. No consultation with anyone other than the instructors are permitted during the exam.
- You have two hours to complete this exam. It is designed to require less than one hour and fifteen minutes of work, so there should not be significant time pressure. Nevertheless, there are many parts to problems, so do not spend too much time on any one question; you can always return to it.

Problem 1 (16 pts)

Answer True or False to each of the questions below. Each question is worth 2 points. Answer true only if it is true as stated, with no additional assumptions. No explanation is needed, but any rational explanation is likely to earn partial credit, and no explanation will not earn any credit if the answer is wrong.

- (a) The best median finding algorithm is on average $\Theta(N)$ but worst case $\Theta(N^2)$.

Solution:

FALSE. The algorithm discussed in class is worst case $\Theta(N)$.

- (b) Given an array of N integers, the best algorithm has to build a min-heap has worst case time $\Theta(N \log N)$.

Solution:

FALSE. The buildheap algorithm discussed in class is worst case $\Theta(N)$.

- (c) For an binary search tree of height H the number of leaves is in $O(2^H)$. (Note $O()$, not $\Theta()$.)

Solution:

True. The maximum number of leaves is $O(2^H)$, so any binary search tree has no more than 2^H leaves.

- (d) $n^2 \log(n) \in O(n^{2.1})$

Solution:

True. Note that $n^{0.1} > \log(n)$ as n increases!

- (e) If $f_i(n) \in O(n^2)$, then $\sum_{i=1}^n (f_i(n))^2 \in O(n^5)$

Solution:

True. There are n terms, at least half of which are $O(n^2)$, so you can bound this by a constant times n^3 , making it $O(n^3)$.

- (f) Inserting numbers $n, n-1, \dots, 1$ one at a time into a binary min-heap in that decreasing order order will take $\Theta(n)$ time.

Solution:

False. It will be $\Theta(n \log(n))$. Note that, since we are inserting them in decreasing order order, there will be $O(\log(n))$ swaps on average, since every inserted key has to upheave all the way to the root.

- (g) The second largest element in a binary min-heap with all elements with distinct values will always be at a leaf.

Solution:

False. Consider the min-heap $[1, 32, 2, 35]$.

- (h) If X is the in-order **successor** of Y , then X has no right child.

Solution:

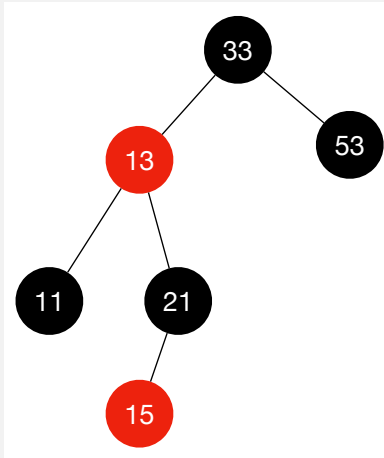
False. No left child.

Problem 2 (25 pts) Quick questions

- (a) The height of a tree is the length of the longest root-to-leaf path in it, in terms of number of edges in the path. What is the minimum number of nodes in a red-black tree of height 3?

Solution:

The minimum number of nodes is 6, illustrated by this tree with black height 2.



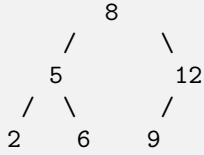
- (b) Consider a complete binary tree with n elements, stored in an array, where the left and the right subtrees of the root are ordered as **max-heaps**. Describe an algorithm with worst case complexity $\Theta(n)$ for converting this tree to a **min-heap**.

Solution:

This is what you get when you do build a heap from any array of n elements using heapify, which is $\Theta(n)$.

- (c) Given a binary search tree, **pre-order** traversal of this tree yields the following sequence: 8,5,2,6,12,9. Draw a complete binary search tree of height 2 consistent with this traversal.

Solution:



- (d) A binary min-heap implemented using an array is given as [8, 10, 16, 19, 18, 17, 20]. What is the content of the array after each of two delete min operations?

Solution:

Delete 1: [8, 10, 16, 19, 18, 17, 20] $\rightarrow$ [10, 18, 16, 19, 20, 17]

Delete 2: [10, 18, 16, 19, 20, 17] $\rightarrow$ [16, 18, 17, 19, 20]

- (e) On a Red-Black binary search tree, assume you are asked to find the lowest common ancestor for two keys a, b in the tree, where all the keys in the tree are unique (no ties). Describe an algorithm for doing this, and provide a bound of the complexity of the algorithm for a tree with n keys.

Solution:

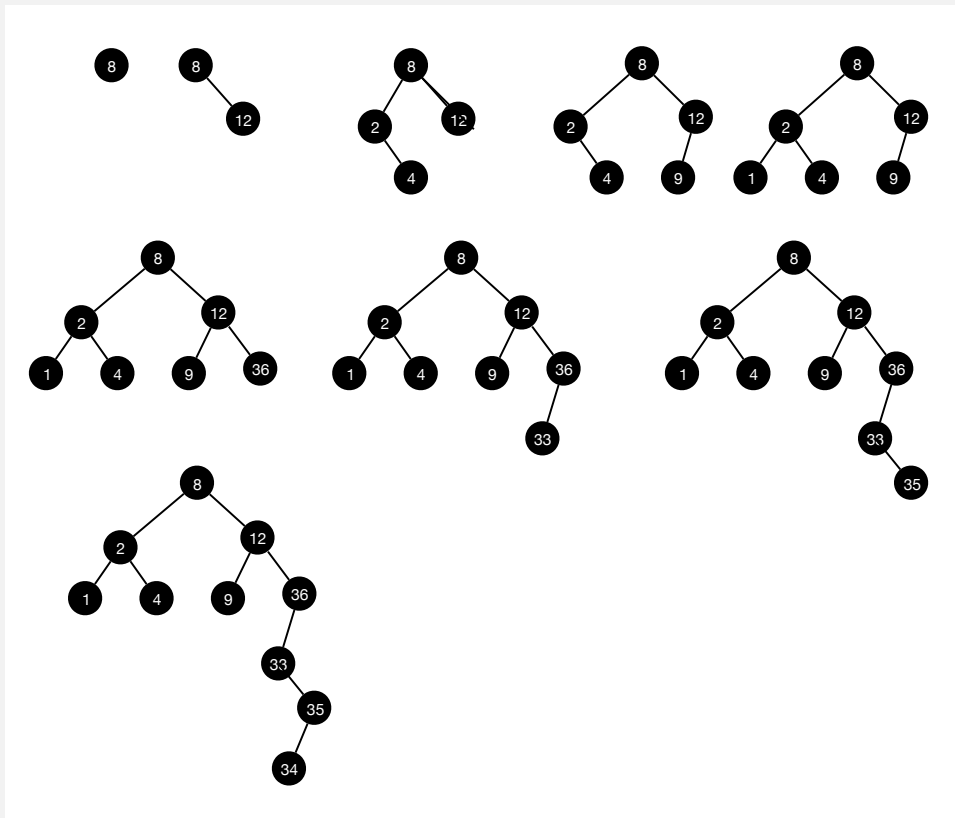
Search the tree until the keys straddle a branch. That is the lowest common ancestor. $O(\log(n))$.

Problem 3 (20 pts) This problem is to construct step by step BST and Red-Black trees given the following list of $N = 10$ elements, shown below:

8, 12, 2, 4, 9, 1, 36, 33, 35, 34

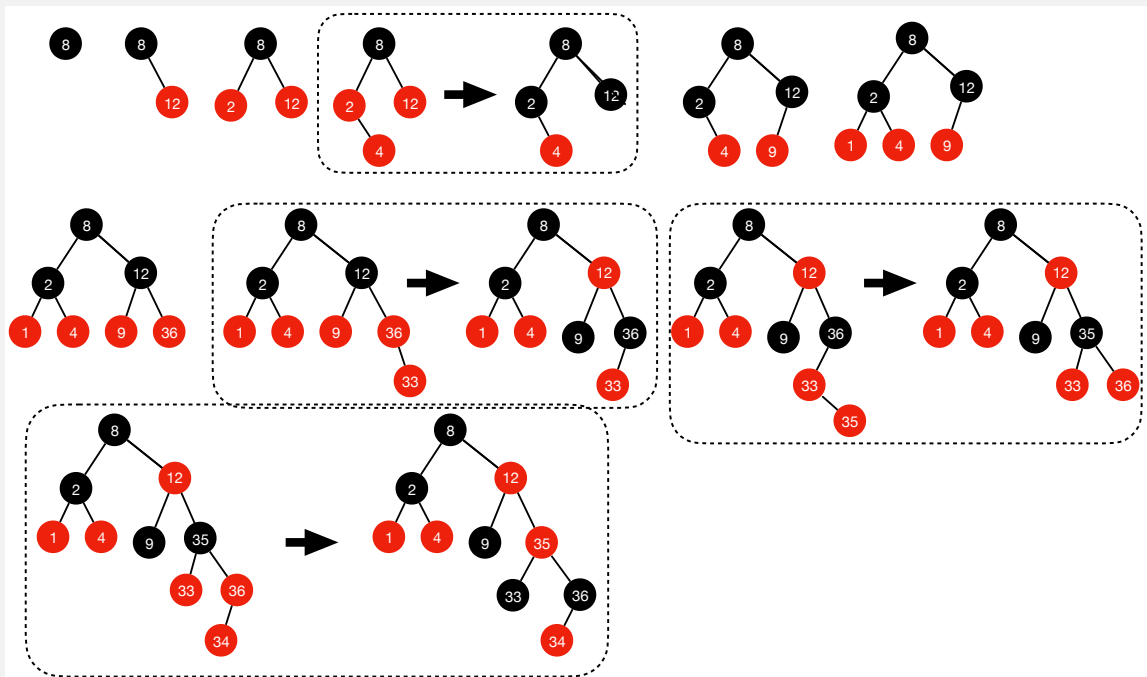
- (a) Insert them sequentially into a Binary Search Tree, showing the sequences of trees that appear after each insert.

Solution:



- (b) Insert them sequentially into a Red-Black Search Tree, restoring the red-black property after each insert. Label the black nodes with a B after each step.

Solution:



Problem 4 (15 pts)

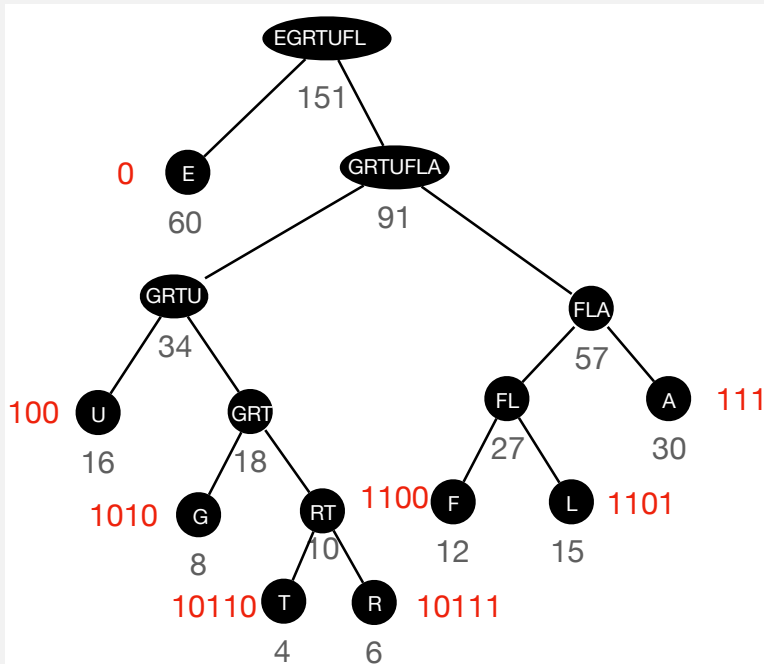
You are interested in compression. Given a file with characters, you want to find the binary code which satisfies the

prefix property (no conflicts) and which minimizes the number of bits required. As an example, consider an alphabet with 8 symbols, with relative weights (frequency) of appearance in an average text file give below:

Alphabet:	G	R	E	A	T	F	U	L
Weights:	8	6	60	30	4	12	16	15

- (a) Determine the Huffman code by constructing a tree that gives the minimum number of characters to encode this document. Arrange the tree so that, when branching, make the left branch the one with the smallest weight.

Solution:



- (b) Identify the code for each letter, using the convention that going left is 0 and going right is 1. List the number of bits for each letter and compute the average number of bits per symbol in this code. (You can leave the answer as a fraction.)

Solution:

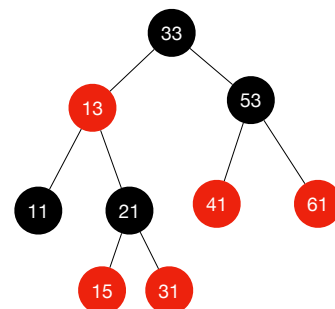
See figure above for code:

E: code 0; 1 bit x 60 occurrences = 60 bits
A: Code 111; 3 bits x 30 occurrences = 90 bits
U: Code 100; 3 bits x 16 occurrences = 48 bits
G: Code 1010; 4 bits x 8 occurrences = 32 bits
F: Code 1100; 4 bits x 12 occurrences = 48 bits
L: Code 1101; 4 bits x 15 occurrences = 60 bits
T: Code 10110; 5 bits x 4 occurrences = 20 bits
R: Code 10111; 5 bits x 6 occurrences = 30 bits

Total bits: 388; Total symbols: 161. Average bits per symbol: $\frac{388}{161}$

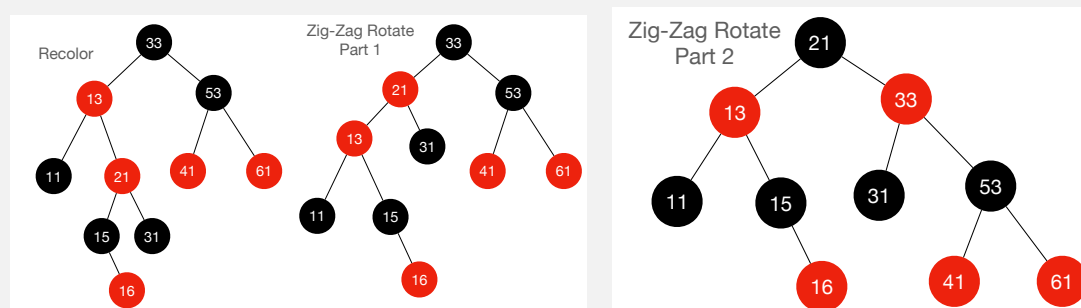
Problem 5 (6 pts)

Consider the red-black tree shown on the right. Show the red-black tree that results when you insert the key 16 into the tree and restore the red-black property.



Solution:

The solution is illustrated below. It involves a recolor, then a zig-zag rotation.



Problem 6 (18 pts)

For each of these recursions, please give the tightest upper bound for the recursion. You can write your answer as $T(n) \in O(g(n))$ for your best choice of function $g(n)$.

(a) $T(n) = 8T(n/2) + n^3$

Solution:

$\log_2(8) = 3$, so the natural growth is n^3 , which is also the order of the forcing function. By the Master theorem, $T(n) \in \Theta(n^3 \log(n)) \in O(n^3 \log(n))$.

(b) $T(n) = 5T(n/2) + (n \log n)^2$

Solution:

$(n \log n)^2 \in O(n^{\log_2(5)-\epsilon})$ for some $\epsilon > 0$, so $T(n) \in \Theta(n^{\log_2(5)})$.

(c) $T(n+1) = 2n + T(n)$, with $T(1) = 1$.

Solution:

$T(n) = 1 + 2 + 4 + 6 + \dots + 2(n-1) = 1 + (n)(n-1) \in \Theta(n^2)$.

(d) $T(n) = 8T(n/2) + n^2$, with $T(1) = 1, T(0) = 0$.

Solution:

$O(n^3)$.

(e) $T(n) = 3T(n/2) + n$.

Solution:

$$O(n^{\log_2(3)})$$

(f) $T(n) = 3T(n/2) + n^{\log_2(3)}.$

Solution:

$$n^{\log_2(3)} = 2^{\log_2(n) \log_2(3)} = 3^{\log_2(n)}$$

$$O(n^{\log_2(3)} \log(n))$$